

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ**

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода классификации данных с использованием
природных временных рядов

Студент гр. 3381
Преподаватель

Кулаков А.Ю.
Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов.

Задание.

1. Подготовить данные. Скачать данные из источника <https://www.nmdb.eu/nest/> за периоды и станции, указанные по вариантам таблиц. Использовать данные с одномоментным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция `median` в MatLab).

2. Сформировать векторы-истории. Временной ряд разбить на последовательные блоки (векторы) длиной m , каждый из которых должен содержать m прошлых значений. Последний вектор в ряду будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчетов.

3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (k) определить равным 2-4.

4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков, нужно взять простое среднее следующих за ними значений.

5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась на языке Python. Исходный код программы приведен в приложении А.

Данные считываются из файла `'./lb2/data.csv'` с разделителем `';`. Для проверки результата загрузки программа выводит первые пять строк:

	<code>start_date_time</code>	<code>RCORR_E</code>
0	2014-11-01 00:00:00	236.34
1	2014-11-01 00:01:00	236.34
2	2014-11-01 00:02:00	237.82
3	2014-11-01 00:03:00	232.96
4	2014-11-01 00:04:00	234.17

После преобразования столбца `start_date_time` в формат даты и времени выводятся размер таблицы и результат `describe()`:

```
(12960, 2)
      start_date_time      RCORR_E
count      12960      12960.000000
mean  2014-11-05 11:59:30      234.220844
min    2014-11-01 00:00:00       0.000000
25%    2014-11-03 05:59:45      231.040000
50%    2014-11-05 11:59:30      233.560000
75%    2014-11-07 17:59:15      236.140000
max    2014-11-09 23:59:00     9164.470000
std                      NaN       78.595767
```

Размер таблицы равен (12960, 2), а значение count для RCORR_E равно 12960. Следовательно, пропущенных значений в столбце RCORR_E нет.

На этапе подготовки данных в программе вычисляются стандартное отклонение и среднее значение RCORR_E, после чего формируется маска `abs(df['RCORR_E'] - mean) > 3 * std`. Результат подсчета записей по этой маске:

```
start_date_time      1
RCORR_E              1
dtype: int64
```

Программой обнаружен 1 выброс. Доля от всех данных рассчитывается выражением `d = 1 / 12960`. Выведенное значение:

```
0.000077
```

После исключения выброса методом `median()` вычисляется медианное значение RCORR_E:

```
233.56
```

Затем найденный выброс заменяется вычисленным медианным значением командой `df.loc[mask, 'RCORR_E'] = med`.

Для формирования векторов-историй в программе задаются `m = 180` и `k = 4`. Векторы создаются функцией `np.lib.stride_tricks.sliding_window_view(data, m)`. Все векторы, кроме последнего, записываются в `X_train`, а последний вектор — в `X_test`.

```
m, k = 180, 4
data = df['RCORR_E']
vectors = np.lib.stride_tricks.sliding_window_view(data, m)
X_train = vectors[:-1]
X_test = vectors[-1:]
```

Для поиска ближайших соседей используются три метрики: 'euclidean', 'manhattan' и 'cosine'. Для каждой метрики вычисляются расстояния от `X_test` до `X_train`, затем выбираются четыре наименьших расстояния.

```
metrics = ['euclidean', 'manhattan', 'cosine']
for metric in metrics:
    dist = pairwise_distances(X_test, X_train, metric=metric).ravel()
    knn = dist.argsort()[:k]
```

Для евклидовой метрики программа выводит:

```
euclidean: расстояния = [61.75596166 62.50122319 62.58511005 62.60607878]
среднее предсказанное значение: 228.70250000000001
```

Для манхэттенской метрики программа выводит:

```
manhattan: расстояния = [637.18 650.78 652.87 653.72]
среднее предсказанное значение: 230.17
```

Для косинусной метрики программа выводит:

```
cosine: расстояния = [0.00016062 0.00017063 0.00017216 0.00017628]
среднее предсказанное значение: 235.655
```

Предсказанное значение для каждой метрики вычисляется в цикле как среднее следующих за четырьмя найденными векторами значений:

```
avg = 0
for ind in knn:
    avg += X_train[ind + m][0] / 4
```

Выводы.

В ходе работы были подготовлены данные и сформированы векторы-истории длиной $m = 180$. Последний вектор использован как тестовый. Число ближайших соседей задано $k = 4$. Расстояния до соседей вычислены с использованием евклидовой, манхэттенской и косинусной метрик. Полученные программой средние предсказанные значения равны 228.70, 230.17 и 235.655 соответственно.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import pairwise_distances

df = pd.read_csv('./lb2/data.csv', sep=';')
print(df.head())
df['start_date_time'] = pd.to_datetime(df['start_date_time'])
print(df.shape)
print(df.describe())
std = df['RCORR_E'].std()
mean = df['RCORR_E'].mean()
mask = (abs(df['RCORR_E'] - mean) > 3 * std)
print((df[mask]).count())

d = 1 / 12960
print(f"{d:.6f}") # доля от всех данных
df_no_anomaly = df[~mask]
med = df_no_anomaly['RCORR_E'].median()
print(med)
df.loc[mask, 'RCORR_E'] = med

m, k = 180, 4
data = df['RCORR_E']
vectors = np.lib.stride_tricks.sliding_window_view(data, m)
X_train = vectors[:-1]
X_test = vectors[-1:]
metrics = ['euclidean', 'manhattan', 'cosine']
for metric in metrics:
    dist = pairwise_distances(X_test, X_train, metric=metric).ravel()
    knn = dist.argsort()[:k]
    print(f"{metric}: расстояния = {dist[knn]}")
    avg = 0
    for ind in knn:
        avg += X_train[ind + m][0] / 4
    print(f"среднее предсказанное значение: {avg}")
```